INSTITUTO POLITÉCNICO NACIONAL

Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas (UPIICSA)

Asignatura: Abstracción Y Uso De Datos

Trabajo: Reporte Integral de Prácticas (1er, 2do y 3er Departamental)

Secuencia: 3CM32

Carrera: Lic. en Ciencias de la Informática

Ruta de Repositorio:

UPIICSA/CI/ABS/2027-1/S32/BreakSyntax at main · alyson-cruz/UPIICSA

Equipo: Break Syntax

Integrantes	**No.**	**Boleta**
Cruz Velasco Alyson Ivanna	4	2026600123
Hernandez Arroyo Miguel Angel	15	2026600419
Ramos Luna Guadalupe Jaquelinee	22	2026600345

Profesor: Yventz Entzana Garduño

Fecha de Entrega: Noviembre de 2026

Índice

1. Desarrollo

1.1 Primer departamental: Fundamentos, POO, arreglos, punteros y recursividad

1.2 Segundo departamental: Métodos de ordenamiento, manejo indirecto y estructuras lineales (adt, pila, cola y lista)

1.3 Tercer departamental: Persistencia en archivos (CSV, XML, TXT, JSON, Binario) y estructuras no lineales (Grafos, dígrafos y árboles)

2. Errores presentados durante el desarrollo

2.1. Primer departamental

2.2. Segundo departamental

3. Solución de los errores

3.1. Soluciones aplicadas en el primer departamental

3.2. Soluciones aplicadas en el segundo departamental

4. Código esencial por departamental

4.1 Primer departamental: Calculadora recursiva con positivos/negativos y manejo de punteros

4.2 Segundo departamental: Ordenamiento indirecto y pila/lista dinámica por referencia

4.3 Tercer departamental: Persistencia de tipos creados/adt en archivos y árbol binario de búsqueda

5. Conclusiones por parcial

5.1 Primer departamental

5.2 Segundo departamental

5.3 Tercer departamental

1. Desarrollo

1.1. Primer departamental

En este primer bloque se construyeron las bases:

- **Fundamentos, sobrecarga y herencia (Prácticas 1 a 6):** Se inició con la impresión de mensajes y el paso de parámetros para sumar dos números. Posteriormente se diseñó una calculadora básica (suma, resta, multiplicación y división) y se aplicó **sobrecarga de métodos** para permitir variantes que reciben 0, 2 o 3 parámetros en la misma clase. Mediante **herencia y sobreescritura**, se creó la clase derivada calculadoranueva que reemplaza los operadores aritméticos directos por algoritmos de sumas sucesivas para la multiplicación, restas sucesivas para la división y multiplicaciones sucesivas para la potencia, adaptando la lógica de signos para admitir números positivos y negativos.
- **Creación de tipos de datos y tamaños en memoria (Prácticas 7 y 7 bis):** Se modelaron las entidades Auto (precio y año) y Persona (nombre, apellido paterno, apellido materno, género y edad) implementando versiones separadas con struct para PE y class para POO. Además, se utilizó el operador sizeof() para inspeccionar y comparar el consumo en bytes de los tipos primitivos frente a las estructuras y clases creadas.
- **Estadística, arreglos, punteros y matrices (Prácticas 8 a 13):** Se programó el cálculo de suma, promedio, media, máximo y mínimo para 5 números usando primero variables individuales, luego un arreglo unidimensional y finalmente acceso indirecto mediante aritmética de punteros (*(p + i)). En el manejo matricial se implementó el producto por una constante y la multiplicación de matrices $A \times B$ dimensionadas por el usuario. Finalmente, se combinaron las estructuras/clases Persona y Auto con arreglos y punteros ((ptr + i)->) para gestionar listas de registros.
- **Recursividad y fractales (Prácticas 14 y 15):** Se sustituyeron los ciclos iterativos por llamadas recursivas para calcular el factorial, la serie de

Fibonacci, las operaciones sucesivas de la calculadora y la generación visual en consola del Triángulo de Sierpinski y el Polvo de Cantor.

1.2. Segundo departamental

El segundo parcial se enfocó en la eficiencia algorítmica para el ordenamiento de información y en la construcción de estructuras de datos lineales estáticas y dinámicas:

- **Algoritmos de ordenamiento directo e indirecto (Prácticas 16 a 21):** Se implementaron los métodos **Burbuja (Bubble Sort)**, **Merge Sort** y **Quick Sort** tanto en PE como en POO para ordenar tres dominios distintos ingresados por el usuario: conjuntos de caracteres, números enteros y el tipo de dato creado (Persona ordenada por edad). En las prácticas 19 a 21 se transformaron estos tres algoritmos para operar de manera indirecta a través de punteros a los arreglos.
- **Autoreferenciación y tipos de datos abstractos (Prácticas 1 y 2 del Bloque 2):** Se modificó la estructura y clase Persona para incluir un puntero al mismo tipo de dato (Persona *siguiente), permitiendo enlazar registros en memoria, y se definió un contenedor ADT dinámico dimensionado por el usuario.
- **Estructuras lineales: pila, cola y lista (Prácticas 3 a 10 del Bloque 2):** Se desarrollaron las tres estructuras fundamentales en tres modalidades distintas:
    1. **De manera estática:** Utilizando arreglos dimensionados por el usuario y controlando los índices de tope, frente y fin.
    2. **De manera dinámica con punteros y referencias:** Creando nodos en tiempo de ejecución (new) que apuntan al mismo tipo de dato y pasando los punteros por referencia (*&) para insertar y liberar memoria (delete) directamente.
    3. **Mediante librerías estándar del IDE:** Empleando las plantillas de la STL de C++ (<stack>, <queue> y <list>).

1.3. Tercer Departamental

El tercer parcial integró la persistencia de datos en almacenamiento secundario y el modelado de estructuras de datos no lineales:

1. **Manejo de archivos en múltiples formatos (Prácticas 11 y 1 a 4 del Bloque 3):** Utilizando la librería <fstream>, se desarrollaron módulos de escritura (ofstream) y lectura (ifstream) para archivos **CSV**, **XML**, **TXT**, **JSON** y **binarios (.dat)**. En cada formato el usuario ingresa datos básicos, arreglos dimensionados, un objeto de tipo creado (Persona o Auto), un arreglo de objetos y un ADT (pila, cola o lista), estructurando las etiquetas o delimitadores correspondientes y reconstruyendo la información en pantalla al leer el archivo.
2. **Grafos y dígrafos (Prácticas 5 y 6 del Bloque 3):** Se representaron relaciones de red mediante matrices de adyacencia dimensionadas según el número de vértices y aristas indicados por el usuario, tanto en PE como en POO. Para los grafos no dirigidos se implementó la simetría de la matriz y el recorrido en anchura (BFS), mientras que en los dígrafos (grafos dirigidos) se incorporaron pesos en los arcos y el cálculo del grado de entrada y salida de cada vértice.
3. **Árboles binarios (Práctica 7 del Bloque 3):** Se programó un Árbol Binario de Búsqueda (ABB) dinámico con nodos enlazados a subárboles izquierdo y derecho, implementando la inserción recursiva, la búsqueda de elementos y los recorridos clásicos en **preorden**, **inorden** y **postorden** en versiones separadas para PE y POO.

2. Errores presentados durante el desarrollo

2.1. Primer departamental

- **Desarrollo inicial en Python en lugar de POO en C++:** Las primeras entregas se realizaron en el lenguaje Python sin implementar la estructura de clases, métodos, sobrecarga ni herencia que exigía el temario.
- **Falta de interactividad con el usuario (ausencia de cin y cout):** Los primeros códigos entregados operaban con datos fijos asignados directamente dentro de las variables, sin utilizar flujos de entrada y salida (cin y cout) para que el usuario ingresara los valores en consola.
- **Fallo al procesar números negativos en las calculadoras (Prácticas 3, 4, 5, 6 y 14):** Las calculadoras iniciales solo funcionaban con números positivos. Al ingresar números negativos en la multiplicación por sumas sucesivas, la división por restas sucesivas, la potencia por multiplicaciones sucesivas o en la calculadora recursiva, las condiciones de parada no se cumplían y provocaban resultados en cero o ciclos infinitos.
- **Matrices con dimensiones fijas no configurables (Práctica 11):** El programa de cálculo matricial ($A \times constante$ y $A \times B$) tenía dimensiones estáticas predefinidas en el código, impidiendo que el usuario dimensionara libremente las filas y columnas de cada matriz y careciendo de validación cuando las matrices no eran conformables.
- **Omisión de los programas con punteros por limitaciones del lenguaje inicial (Prácticas 10 y 13):** Debido a que los primeros ejercicios se hicieron en Python, no se logró simular el manejo de direcciones de memoria ni la aritmética de punteros, por lo que no se pudieron entregar las prácticas de promedio con puntero al arreglo ni el acceso indirecto a la lista de personas.
- **Entrega de PE y POO en un mismo archivo (Prácticas 7, 7 bis, 12 y 13):** En las prácticas que solicitaban crear tipos de datos (Auto y Persona), identificar tamaños con sizeof y manejar arreglos tanto en Programación Estructurada (PE) como en Programación Orientada a Objetos (POO), se colocaron ambos códigos dentro de un solo archivo. Esto generó conflictos de redefinición de nombres y errores en la función principal, provocando que fallara la parte de PE o fallara la parte de POO.

2.2. Segundo departamental

- **Falta de entrega de ordenamientos indirectos y estructuras con punteros (Prácticas 19, 20, 21 y Bloque 2):** Al intentar mantener el enfoque de Python del parcial anterior, no fue posible implementar el ordenamiento de manera indirecta (Burbuja, Merge Sort y Quick Sort con punteros) ni los enlaces dinámicos con punteros al mismo tipo de dato y referencias para pilas, colas y listas.
- **Conflicto al unir PE y POO en los métodos de ordenamiento (Prácticas 16 a 21 y Práctica 1 del Bloque 2):** Los algoritmos de ordenamiento Burbuja, Merge Sort y Quick Sort para caracteres, enteros y tipos de datos creados, así como el manejo indirecto con estructuras y clases autoreferenciadas, se habían integrado en un único archivo por práctica. Al compilar, las estructuras de PE interferían con las clases de POO, impidiendo que ambos enfoques funcionaran correctamente al mismo tiempo.
- **Falta de dimensionamiento e interacción en estructuras lineales y ordenamientos:** En las primeras versiones de los métodos de ordenamiento y de las estructuras estáticas (ADT, pila, cola y lista por medio de arreglos), el usuario no podía definir la cantidad de elementos a ordenar ni la capacidad máxima de las estructuras mediante cin y cout.

3. Solución de los errores

3.1. Soluciones aplicadas en el primer departamental

1. **Migración total a C++ bajo el paradigma POO:** Se reescribieron las 15 prácticas desde cero en C++ utilizando clases (`class`), declarando todos sus atributos y métodos.
2. **Incorporación de entrada y salida interactiva (`cin` y `cout`):** En cada uno de los programas se agregaron instrucciones `cout` para solicitar datos claros al usuario y lecturas con `cin` para que toda la información (nombres, edades, precios, números de prueba y niveles de fractales) sea ingresada en tiempo de ejecución.

3. **Soporte completo para números positivos y negativos en las calculadoras:** En las prácticas 3 a 6 y 14 se convirtieron los números ingresados a su valor absoluto (`int posA = (a < 0) ? -a : a;`) antes de realizar las sumas, restas o multiplicaciones sucesivas e iteraciones recursivas. Al finalizar el cálculo se aplicó la regla de los signos mediante la condición `if ((a < 0 && b > 0) || (a > 0 && b < 0))` para devolver el resultado negativo cuando corresponde, y en potencias con exponente negativo se calculó la fracción inversa `1.0f / resultado`.
4. **Dimensionamiento dinámico de matrices por el usuario:** En la práctica 11 se añadieron variables (`fA`, `cA`, `fB`, `cB`) leídas por teclado con `cin` para que el usuario establezca las dimensiones exactas de las matrices AAA y BBB antes de capturar sus valores, incluyendo la validación `if (cA != fB)` para verificar que la multiplicación sea matemáticamente realizable.
5. **Implementación nativa de punteros en C++:** Al programar directamente en C++, se resolvieron las prácticas 10 y 13 declarando punteros explícitos (`float *p = nums;` y `Persona *ptr = lista;`), recorriendo y modificando las direcciones de memoria de forma indirecta mediante desreferenciación `*(p + i)` y el operador flecha `(ptr + i)->`.
6. **Separación de PE y POO en dos archivos independientes:** Para las prácticas 7, 7 bis, 12 y 13 se dividió cada entrega en dos códigos `.cpp` completamente separados: uno exclusivo para Programación Estructurada usando `struct` y funciones libres, y otro exclusivo para Programación Orientada a Objetos usando `class` y métodos internos, eliminando cualquier conflicto de compilación.

3.2. Soluciones aplicadas en el segundo departamental

1. **Separación en archivos independientes para PE y POO en todos los ordenamientos (Prácticas 16 a 21 y Práctica 1 del Bloque 2):** Se crearon dos programas separados por cada método de ordenamiento (Burbuja, Merge Sort y Quick Sort, tanto directos como indirectos): el primer código

implementado con `struct Persona` y funciones estructuradas, y el segundo encapsulado dentro de clases (`OrdenamientoBurbuja`, `MergeSortPOO`, `QuickSortPOO`).

2. **Implementación de punteros, memoria dinámica (`new`/`delete`) y paso por referencia (`*&`):**
    - En las prácticas 19, 20 y 21 se utilizaron punteros (`char *pL`, `int *pN`, `Persona *pP`) para comparar e intercambiar las posiciones de memoria de manera indirecta durante los ordenamientos.
    - En la práctica 1 del segundo bloque se incluyó el puntero autoreferenciado `Persona *siguiente;` dentro de la estructura y de la clase para enlazar los objetos secuencialmente.
    - En las prácticas 6, 7 y 7 bis se implementaron nodos dinámicos enlazados pasando los punteros por referencia (`NodoPila *&tope`, `NodoCola *&frente`, `NodoLista *&cabeza`), logrando que las inserciones y eliminaciones actualicen la memoria real sin perder el rastro de la estructura.
3. **Dimensionamiento por el usuario y menús interactivos en estructuras de datos:** Tanto en los métodos de ordenamiento como en el ADT, pila, cola y lista (estáticas, dinámicas y con librerías `<stack>`, `<queue>` y `<list>`), se agregó al inicio la lectura con `cin` para que el usuario defina cuántos elementos o qué capacidad tendrá la estructura, seguido de un menú interactivo con `while` para insertar, eliminar y mostrar los datos en todo momento.

4. Código esencial por departamental

4.1. Primer Departamental: Calculadora Sucesiva/Recursiva con Signos y Aritmética de Punteros

C/C++

```
// Fragmento esencial: Multiplicacion y division recursiva
aceptando positivos y negativos

class calculadorarecursiva : public calculadora {

public:

    int multPositivos(int a, int b) {

        if (b == 0) return 0;

        return sumar(a, multPositivos(a, b - 1));

    }



    int multiplicar(int a, int b) {

        int posA = (a < 0) ? -a : a;

        int posB = (b < 0) ? -b : b;

        int res = multPositivos(posA, posB);

        return ((a < 0 && b > 0) || (a > 0 && b < 0)) ? -res :
res;

    }



    int divPositivos(int a, int b) {

        if (a < b) return 0;

        return sumar(1, divPositivos(restar(a, b), b));
```

```
    }


    int dividir(int a, int b) {
        if (b == 0) return 0;
        int posA = (a < 0) ? -a : a;
        int posB = (b < 0) ? -b : b;
        int res = divPositivos(posA, posB);
        return ((a < 0 && b > 0) || (a > 0 && b < 0)) ? -res :
res;
    }
};
// Fragmento esencial: Acceso indirecto a arreglos de objetos
mediante puntero (Practica 13)
Persona lista[50];
Persona *ptr = lista;
for (int i = 0; i < n; i++) {
    (ptr + i)->capturar();
}
```

4.2. Segundo Departamental: Ordenamiento Indirecto y Nodos Dinámicos con Referencia

C/C++

```
// Fragmento esencial: Quick Sort indirecto con punteros sobre tipo de dato creado (Practica 21)

int ordenarPersona(Persona *p, int inicio, int fin) {

    int i = inicio, j = fin;

    int pivote = (p + (inicio + fin) / 2)->edad;

    while (i <= j) {

        while ((p + i)->edad < pivote) i++;

        while ((p + j)->edad > pivote) j--;

        if (i <= j) {

            Persona aux = *(p + i);

            *(p + i) = *(p + j);

            *(p + j) = aux;

            i++;

            j--;

        }

    }

    if (inicio < j) ordenarPersona(p, inicio, j);

    if (i < fin) ordenarPersona(p, i, fin);

    return 1;

}
```

```
// Fragmento esencial: Manejo dinamico con puntero al mismo tipo de dato y referencia (Practicas 6, 7 y 7 bis)

class NodoPila {

public:

    int dato;

    NodoPila *siguiente;

};



int push(NodoPila *&tope, int valor) {

    NodoPila *nuevo = new NodoPila();

    nuevo->dato = valor;

    nuevo->siguiente = tope;

    tope = nuevo;

    return 1;

}
```

4.3. **Tercer Departamental:** Persistencia de Archivos y Árbol Binario de Búsqueda

C/C++

```
// Fragmento esencial: Escritura y lectura de estructura JSON con datos basicos, arreglos, objetos y ADT (Practica 3)
```

```
ofstream jsonOut("datos.json");

jsonOut << "{\n  \"datoBasico\": " << basico << ",\n";

jsonOut << "  \"persona\": {\"nombre\": \"" << p.nombre << "\",
\"edad\": " << p.edad << "}\n}" << endl;

jsonOut.close();



ifstream jsonIn("datos.json");

string linea;

while (getline(jsonIn, linea)) {

    cout << linea << endl;

}

jsonIn.close();

// Fragmento esencial: Insercion y recorrido Inorden en Arbol
Binario de Busqueda (Practica 7)

int insertar(NodoArbol *&nodo, int valor) {

    if (nodo == NULL) {

        nodo = new NodoArbol();

        nodo->dato = valor;

        nodo->izq = NULL;

        nodo->der = NULL;
```

```
        return 1;
    }
    if (valor < nodo->dato) return insertar(nodo->izq, valor);
    return insertar(nodo->der, valor);
}


int inorden(NodoArbol *nodo) {
    if (nodo == NULL) return 0;
    inorden(nodo->izq);
    cout << nodo->dato << " ";
    inorden(nodo->der);
    return 1;
}
```

5. Conclusiones

5.1. Primer departamental

Durante el primer departamental se comprendió en la práctica la diferencia estructural y conceptual entre la Programación Estructurada (struct) y la Programación Orientada a Objetos (class). La implementación de sobrecarga de funciones, herencia y sobreescritura en el caso de estudio de la calculadora permitió reutilizar código sin duplicar las operaciones elementales, demostrando cómo descomponer operaciones complejas (multiplicación, división y potencia) en sumas y restas elementales. Asimismo, el trabajo con arreglos dimensionados por el

usuario, el operador sizeof, la aritmética de punteros y la recursividad (tanto numérica como gráfica en los fractales de Sierpinski y Cantor) consolidó el dominio sobre cómo se organiza y direcciona la memoria en C++.

5.2. Segundo departamental

En el segundo departamental se comprobó la importancia de elegir el algoritmo de ordenamiento y la estructura de datos adecuada según el problema a resolver. Al programar Burbuja, Merge Sort y Quick Sort de forma directa e indirecta mediante punteros, se observó cómo los métodos recursivos de "divide y vencerás" optimizan el ordenamiento tanto de datos primitivos como de objetos compuestos (Persona). Por otra parte, construir las estructuras lineales (Pila, Cola y Lista) en sus tres variantes, estática con arreglos, dinámica mediante nodos autoreferenciados con paso por referencia (*&) y dinámica mediante las librerías STL del IDE; permitió entender el funcionamiento interno de la memoria dinámica (new y delete) antes de utilizar contenedores preconstruidos.

5.3. Tercer departamental

El tercer departamental permitió superar la volatilidad de la memoria RAM mediante la persistencia de datos en archivos externos. Escribir y leer tipos de datos básicos, arreglos, objetos y estructuras ADT en formatos estándar de la industria como CSV, XML, TXT, JSON y archivos binarios dio una visión clara de cómo los programas almacenan e intercambian información estructurada entre distintas plataformas. Finalmente, la implementación de estructuras no lineales (Grafos, Dígrafos y Árboles Binarios de Búsqueda) integró los conceptos de matrices, punteros y recursividad vistos en los parciales anteriores para modelar redes de conexiones, calcular grados de incidencia y realizar búsquedas y recorridos jerárquicos de manera eficiente.